Ejercicios de Práctica Avanzados - Vol. 2
Spring Boot & Thymeleaf

Nuevos Ejercicios Prácticos por Bloques con Solucionario al Final

PARTE 1: ENUNCIADOS DE LOS EJERCICIOS (NUEVO SET)

> *Instrucciones: Intenta resolver estos nuevos ejercicios en tu IDE antes de bajar al solucionario del final.*

Bloque 1: Controladores y Parámetros en Spring Boot

Ejercicio 1.2:
1. Crea una clase **CursoController** anotada con **@Controller** y ruta base **/academico**.
2. Define una ruta GET **/filtrar** que reciba un parámetro opcional u obligatorio **nivel** mediante **@RequestParam**. Pásalo al modelo como **"nivelFiltro"** y devuelve la vista **"lista-cursos"**.
3. Define una ruta GET **/curso/{codigo}/modulo/{idModulo}** que reciba dos variables de ruta **@PathVariable** (**String codigo** e **int idModulo**). Pásalas al modelo y devuelve la vista **"detalle-modulo"**.

Bloque 2: Estructuras de Datos Complejas en el Modelo

Ejercicio 2.2:
1. Crea la controladora **EmpresaController** con la ruta **/dashboard**.
2. Envía al modelo un **Map<String, String>** llamado **"empleadosRoles"** utilizando **Map.of("Carlos", "Desarrollador", "Laura", "Diseñadora", "Marcos", "Project Manager")**.
3. En la plantilla **dashboard.html**, recorre el mapa usando **th:each="entrada : ${empleadosRoles}"** y muestra cada nombre (clave) y su puesto (valor) en una lista desordenada.

Bloque 3: Condicionales Complejos y Bucles

Ejercicio 3.3:
Dada una lista de tareas **${tareas}** con propiedades **titulo**, **prioridad** ("ALTA", "MEDIA", "BAJA") y **completada** (boolean):
1. Recorre la lista filtrando únicamente las tareas NO completadas usando **th:unless="${tarea.completada}"** en el mismo elemento o en combinación con **th:each**.
2. Muestra un indicador de prioridad según el caso con **th:switch="${tarea.prioridad}"**.

Bloque 4: Utilitarios de Cadenas y Colecciones

Ejercicio 4.2:
Escribe expresiones de Thymeleaf para realizar lo siguiente:
1. Obtener la longitud de una cadena **${descripcion}** usando **#strings.length(...)**.

2. Comprobar si la lista **${alumnos}** está vacía usando **#lists.isEmpty(...)** dentro de un **th:if** para mostrar el mensaje "No hay alumnos matriculados".
3. Obtener la cantidad total de elementos en la lista **${alumnos}** usando **#lists.size(...)**.

Bloque 5: Combinación de Variables Locales y Bloques

Ejercicio 5.3:
En una tabla de calificaciones, recibes del modelo una lista de alumnos donde cada objeto tiene **nombre** y **nota**.
1. Usa **<th:block th:each="alumno : ${alumnos}">** para agrupar las filas sin introducir etiquetas extra.
2. Dentro del bloque, define con **th:with="aprobado=${alumno.nota >= 5}"** una variable local en la fila <tr>.
3. Muestra "Aprobado" si **aprobado** es true o "Suspenso" si es false.

Bloque 6: Fragmentos y Encabezados Dinámicos

Ejercicio 6.2:
1. Crea un fragmento llamado **cabecera(titulo, subtitulo)** en **src/main/resources/templates/fragmentos/componentes.html**.
2. Desde una plantilla **contacto.html**, incluye el fragmento pasando valores recibidos del modelo **${tituloPagina}** y **${subtituloPagina}** con **th:replace**.

PARTE 2: SOLUCIONARIO Y CÓDIGO RESUELTO (VOL. 2)

Solución - Bloque 1: Controladores y Parámetros Múltiples

```
package com.ejemplo.demo.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
@RequestMapping("/academico")
public class CursoController {

    // Ruta: /academico/filtrar?nivel=AVANZADO
    @GetMapping("/filtrar")
    public String filtrarCursos(@RequestParam String nivel, Model model) {
        model.addAttribute("nivelFiltro", nivel);
        return "lista-cursos";
    }

    // Ruta: /academico/curso/DAW2026/modulo/105
    @GetMapping("/curso/{codigo}/modulo/{idModulo}")
    public String verModulo(@PathVariable String codigo,
                            @PathVariable int idModulo,
                            Model model) {
        model.addAttribute("codigoCurso", codigo);
        model.addAttribute("idModulo", idModulo);
        return "detalle-modulo";
    }
}
```

Solución - Bloque 2: Recorrer Mapas en Thymeleaf

Controlador Java (EmpresaController.java):

```
package com.ejemplo.demo.controller;

import java.util.Map;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class EmpresaController {
```

```
    @GetMapping("/dashboard")
    public String mostrarDashboard(Model model) {
        Map<String, String> empleados = Map.of(
            "Carlos", "Desarrollador",
            "Laura", "Diseñadora",
            "Marcos", "Project Manager"
        );
        model.addAttribute("empleadosRoles", empleados);
        return "dashboard";
    }
}
```

Plantilla Thymeleaf (dashboard.html):

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="es">
<head>
    <meta charset="UTF-8">
    <title>Dashboard Empresa</title>
</head>
<body>
    <h1>Plantilla de Empleados</h1>
    <ul>
        <!-- Iteración sobre Map: entrada.key y entrada.value -->
        <li th:each="entrada : ${empleadosRoles}">
            <strong th:text="${entrada.key}">Nombre</strong>:
            <span th:text="${entrada.value}">Puesto</span>
        </li>
    </ul>
</body>
</html>
```

Solución - Bloque 3: Filtrado y Switch de Tareas

```
<ul>
    <li th:each="tarea : ${tareas}" th:unless="${tarea.completada}">
        <span th:text="${tarea.titulo}">Título Tarea</span>

        <!-- Evaluar prioridad con th:switch -->
        <span th:switch="${tarea.prioridad}">
            <strong th:case="'ALTA'" style="color: red;">[¡URGENTE!]</strong>
            <strong th:case="'MEDIA'" style="color: orange;">[Normal]</strong>
            <strong th:case="'BAJA'" style="color: green;">[Baja]</strong>
            <strong th:case="*">[Sin prioridad]</strong>
        </span>
    </li>
</ul>
```

Solución - Bloque 4: Utilitarios #strings y #lists

```
<div>
    <!-- 1. Longitud de cadena -->
    <p>Caracteres en la descripción: <span
th:text="${#strings.length(descripcion)}">0</span></p>

    <!-- 2. Condicional comprobando si la lista está vacía -->
    <p th:if="${#lists.isEmpty(alumnos)}" style="color: red;">
        No hay alumnos matriculados.
    </p>

    <!-- 3. Tamaño total de la lista -->
    <p th:unless="${#lists.isEmpty(alumnos)}">
        Total de alumnos matriculados: <span
th:text="${#lists.size(alumnos)}">0</span>
    </p>
</div>
```

Solución - Bloque 5: th:block con th:with en Tabla

```
<table border="1">
    <thead>
        <tr>
            <th>Alumno</th>
            <th>Nota</th>
            <th>Estado</th>
        </tr>
    </thead>
    <tbody>
        <!-- Agrupación con th:block sin alterar contenedor -->
        <th:block th:each="alumno : ${alumnos}">
            <tr th:with="aprobado=${alumno.nota >= 5}">
                <td th:text="${alumno.nombre}">Nombre</td>
                <td th:text="${alumno.nota}">0.0</td>
                <td>
                    <span th:if="${aprobado}" style="color: green;">Aprobado</span>
                    <span th:unless="${aprobado}" style="color: red;">Suspenso</span>
                </td>
            </tr>
        </th:block>
    </tbody>
</table>
```

Solución - Bloque 6: Fragmento Cabecera con Parámetros del Modelo

Fragmento (fragmentos/componentes.html):

```
<!DOCTYPE html>
```

```
<html xmlns:th="http://www.thymeleaf.org" lang="es">
<body>

    <!-- Fragmento de cabecera con dos parámetros -->
    <header th:fragment="cabecera(titulo, subtitulo)">
        <h1 th:text="${titulo}">Título por defecto</h1>
        <p th:text="${subtitulo}">Subtítulo por defecto</p>
        <hr>
    </header>

</body>
</html>
```

Inclusión desde Vista (contacto.html):

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="es">
<head>
    <meta charset="UTF-8">
    <title>Contacto</title>
</head>
<body>

    <!-- Inclusión pasando atributos enviados desde el modelo Java -->
    <div th:replace="~{fragmentos/componentes :: cabecera(${tituloPagina},
${subtituloPagina})}"></div>

    <main>
        <p>Formulario de contacto y soporte de la aplicación.</p>
    </main>

</body>
</html>
```